



FRAMEWORK NÃO INVASIVO PARA EXTENSÃO DE JANELA DE CONTEXTO EM LARGE LANGUAGE MODELS COM RECUPERAÇÃO DE INFORMAÇÕES E RACIOCÍNIO SOCRÁTICO

Matheus Cardoso

Dissertação de Mestrado apresentada ao Programa de Pós-graduação em Engenharia de Sistemas e Computação, COPPE, da Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Mestre em Engenharia de Sistemas e Computação.

Orientadores: Geraldo Xexeo
Claudia Susie

Rio de Janeiro
Fevereiro de 2011

FRAMEWORK NÃO INVASIVO PARA EXTENSÃO DE JANELA DE
CONTEXTO EM LARGE LANGUAGE MODELS COM RECUPERAÇÃO DE
INFORMAÇÕES E RACIOCÍNIO SOCRÁTICO

Matheus Cardoso

DISSERTAÇÃO SUBMETIDA AO CORPO DOCENTE DO INSTITUTO
ALBERTO LUIZ COIMBRA DE PÓS-GRADUAÇÃO E PESQUISA DE
ENGENHARIA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO
GRAU DE MESTRE EM CIÊNCIAS EM ENGENHARIA DE SISTEMAS E
COMPUTAÇÃO.

Orientadores: Geraldo Xexeo
Claudia Susie

Aprovada por: Prof. Nome do Primeiro Examinador Sobrenome

RIO DE JANEIRO, RJ – BRASIL
FEVEREIRO DE 2011

Cardoso, Matheus

Framework Não Invasivo para Extensão de Janela de Contexto em Large Language Models com Recuperação de Informações e Raciocínio Socrático/Matheus Cardoso. – Rio de Janeiro: UFRJ/COPPE, 2011.

XI, 37 p.: il.; 29, 7cm.

Orientadores: Geraldo Xexeo

Claudia Susie

Dissertação (mestrado) – UFRJ/COPPE/Programa de Engenharia de Sistemas e Computação, 2011.

Referências Bibliográficas: p. 33 – 37.

1. Primeira palavra-chave. 2. Segunda palavra-chave. 3. Terceira palavra-chave. I. Xexeo, Geraldo *et al.* II. Universidade Federal do Rio de Janeiro, COPPE, Programa de Engenharia de Sistemas e Computação. III. Título.

a quem o valor de ser dedicado

Agradecimentos

Agradecer primeiramente a Deus, sem ele nada seria possível. Meus orientadores por todo apoio técnico nessa vida academica. A minha esposa por todo suporte e apoio na jornada. A minha familia com apoio incondicional durante todo o processo. Por ultimo a minha cadelinha Jessie com todo apoio emocional durante o processo de escrita dessa dissertação

Resumo da Dissertação apresentada à COPPE/UFRJ como parte dos requisitos necessários para a obtenção do grau de Mestre em Ciências (M.Sc.)

FRAMEWORK NÃO INVASIVO PARA EXTENSÃO DE JANELA DE
CONTEXTO EM LARGE LANGUAGE MODELS COM RECUPERAÇÃO DE
INFORMAÇÕES E RACIOCÍNIO SOCRÁTICO

Matheus Cardoso

Fevereiro/2011

Orientadores: Geraldo Xexeo
Claudia Susie

Programa: Engenharia de Sistemas e Computação

Apresenta-se, nesta tese, ...

Abstract of Dissertation presented to COPPE/UFRJ as a partial fulfillment of the requirements for the degree of Master of Science (M.Sc.)

NON-INVASIVE CONTEXT WINDOW EXTENSION FRAMEWORK FOR
LARGE LANGUAGE MODELS WITH INFORMATION RETRIEVAL AND
SOCRATIC REASONING

Matheus Cardoso

February/2011

Advisors: Geraldo Xexeo
Claudia Susie

Department: Systems Engineering and Computer Science

In this work, we present ...

Sumário

Lista de Figuras	x
Lista de Tabelas	xi
1 Introdução	1
1.1 Contextualização	1
1.1.1 O Problema Crítico: Janela de Contexto versus Desempenho .	3
1.2 Problema de Pesquisa	3
1.3 Objetivos	4
1.3.1 Objetivo Global	4
1.3.2 Objetivos Específicos	4
1.4 Justificativa e Contribuições	5
1.4.1 Principais Contribuições	6
1.5 Estrutura da Dissertação	7
2 Fundamentos Técnicos e Teóricos	9
2.1 Fundamentação Teórica: Método Socrático	9
2.1.1 O Método Socrático	9
2.1.2 Elenchos como refutação	11
2.1.3 Contraexemplo como ferramenta epistemológica	12
2.1.4 Adaptação do elenchos para avaliação de LLMs	12
2.2 Fundamentos Técnicos de LLMs	13
2.2.1 Modelos de Linguagem de Grande Escala	13
2.2.2 Mecanismo de Atenção e Janela de Contexto	13
2.2.3 Complexidade Computacional Quadrática	14
2.3 Estratégias Não-Invasivas para Extensão de Contexto	15
2.3.1 Compressão de Prompt	16
2.3.2 Segmentação de Contexto e Janela Deslizante	16
2.3.3 Métodos Baseados em MapReduce	17
2.3.4 Alocação e Gerenciamento de Memória Externa	18
2.3.5 Síntese: Taxonomia de Técnicas Não-Invasivas	22

2.3.6	Avaliação de Eficácia: Métricas e Resultados	22
2.3.7	Benchmarks Especializados para Contexto Longo	24
2.3.8	Síntese e Lacunas Identificadas	25
3	Metodologia	26
3.1	Validação Empírica de Técnicas Não-Invasivas	26
3.1.1	Estratégias de Avaliação	26
3.1.2	Resultados: Acurácia e Latência	27
3.1.3	Discussão dos Resultados e Seleção da Estratégia	29
3.2	Arquitetura do Framework Prospoto	29
4	Resultados e Discussões	31
5	Conclusão	32
	Referências Bibliográficas	33

Lista de Figuras

2.1	Fluxo das funções dialéticas, maiêuticas e elênticas no diálogo socrático [1–3].	10
3.1	Diagrama de fluxo do framework de diálogo com extensão de contexto. O sistema integra recuperação via RIG (Dartboard Ranking), compressão adaptativa e geração via LLM.	30

Lista de Tabelas

2.1	Etapas do método socrático no contexto da dissertação	11
2.2	Taxonomia de técnicas não-invasivas para extensão de contexto em LLMs	23
3.1	Acurácia por estratégia nos <i>benchmarks</i> de contexto longo	27
3.2	Latência média por estratégia (ms)	28

Capítulo 1

Introdução

1.1 Contextualização

Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs) mudaram de uma forma decisiva todo o campo de processamento de linguagem natural (*Natural Language Processor* - NLP). Antes da popularização dos LLMs, grande parte das abordagens em NLP era baseada em pipelines rígidos, apoiados em técnicas como *tokenization*, *stemming*, *lemmatization*, *TF-IDF* e remoção de *stopwords*, que buscavam transformar texto bruto em representações numéricas esparsas, como *Bag-of-Words*, para então alimentar modelos estatísticos ou algoritmos clássicos de aprendizado de máquina. Essas abordagens dependiam fortemente de engenharia manual de atributos (*feature engineering*) e ajuste fino (*fine-tuning*) específico para cada conjunto de dados, dificultando bastante a generalização para novos domínios.

A introdução de representações distribuídas, como **Word2Vec** em 2013 [4], começou a mudar o panorama ao permitir que modelos neurais aprendessem, de forma automática, vetores densos que capturavam dos texto as relações semânticas e sintáticas entre palavras, reduzindo assim a necessidade das etapas descritas anteriormente de normalização dos textos. No entanto, foi a adoção da arquitetura de *Transformers*, proposta por Vaswami et al. em 2017 [5] que realmente revolucionou o campo. Modelos baseados nessa arquitetura, como **BERT** [6] e **GPT** [7], introduziram o novo paradigma da área: treinamento prévio com grandes volumes de dados de texto não estruturados junto com o ajuste fino em tarefas específicas com conjuntos de dados menores e rotulados. Esse paradigma de aprendizado por transferência (*transfer learning*) [8] permitiu que representações contextualizadas fossem aprendidas diretamente de textos comuns do dia a dia, eliminando a necessidade de padronização rígida dos dados de entrada e elevando significativamente o desempenho em diversas tarefas.

Com a disponibilização comercial do ChatGPT em novembro de 2022, esses

modelos deixaram de ser apenas objetos de pesquisa para se tornarem componentes centrais em sistemas que realizam, com desempenho consideravelmente bom, tarefas de análise de sentimento, classificação de textos, extração de informações específicas de documentos e apoio à programação [6, 9]. Modelos mais recentes, como a família GPT, Claude, Gemini e LLaMa, são hoje integrados a assistentes virtuais (como Siri da Apple, Copilot da Microsoft), fluxos de análise e produção textual (ChatGPT, Google Chat, DeepSeek Chat) e ambientes de desenvolvimento de software (Cursor, Copilot, Windsurf), tornando-se parte de uma infraestrutura de base em aplicações modernas nas áreas financeira, da saúde, programação e educação [10, 11]. Apesar desse avanço, todos esses modelos ainda esbarram em uma limitação estrutural fundamental: a **Janela de Contexto**.

A janela de contexto corresponde ao número máximo de tokens que um modelo consegue processar de *input* na requisição, funcionando como uma memória de trabalho disponível durante a interação. À medida que esse limite aumenta, maior o custo computacional, pois o mecanismo de atenção apresenta complexidade quadrática $O(n^2)$ no comprimento da sequência. Isso acontece pois cada token é comparado com todos os demais da sequência de texto. Isso faz com que tanto o consumo de memória em GPUs quanto o tempo de processamento do modelo aumentem de forma acentuada conforme a janela de contexto se expande [5, 12]. Na prática, essa restrição impacta diretamente a viabilidade de aplicações que exigem interações longas com o usuário, como análise de grandes documentos, inspeção de grandes volumes de logs ou processamento de múltiplos arquivos em uma única chamada.

Historicamente, houve um aumento significativo no tamanho das janelas de contexto em Modelos baseados em *Transformers*. Em 2018-2019, o **BERT** apresentava janela de contexto de 512 tokens. Em 2023 o **GPT-4 Turbo** conseguiu ampliar esse limite para 128.000 tokens. Em 2024, arquiteturas como **Gemini 1.5** e **Llama 3.1** romperam a barreira de um milhão de tokens. Nos últimos lançamentos, o foco parece ter migrado de ampliar a janela para melhorar qualidade e eficiência das respostas através do aprimoramento do *reasoning*. Isso pode notar no lançamento do modelo **Gemini 3** [13]. Essa expansão, embora impressionante, não eliminou os problemas atrelados a ela. De fato, novos desafios surgiram: além da complexidade computacional já mencionada, fenômenos como degradação de desempenho com aumento do contexto (*context rot*) [14] e o colapso da atenção (*rank-collapse*), onde as pontuações de atenção tendem à uniformidade, comprometem a eficácia dos modelos em cenários de contexto genuinamente extenso [15].

1.1.1 O Problema Crítico: Janela de Contexto versus Desempenho

Apesar da capacidade teórica de processamento de milhões de tokens, os modelos modernos frequentemente sofrem com alucinações por falta de uso do contexto de forma eficaz. Estudos mais recentes [14, 15] relatam a degradação do desempenho dos modelos a maneira que mais informação é inserida na janela de contexto. Essa limitação decorre de um recurso de atenção finito, cujos retornos diminuem conforme novos tokens são adicionados. De forma análoga a memória humana, os LLMs enfrentam escassez de atenção intrínseca à sua arquitetura. Cada token adicionado consome parte do "orçamento de atenção", dificultando a priorização de informações importantes.

Além disso, surgem desafios práticos e econômicos associados ao uso de contexto extensos:

- **Viés “Lost in the Middle“:** Tokens posicionados no início ou no final da sequência tendem a receber mais atenção enquanto aqueles situados na região central tendem a ser ignorados. Liu et al. [16] mostram que o desempenho dos LLMs sofre uma curva em formato de "U", onde a informação no meio de grandes documentos frequentemente é ignorada, ocasionando em alucinações.
- **Custo Computacional e Econômico:** Como o processamento de tokens é o principal componente de custo de processamento dos modelos em servidores, o uso de contextos muito longos pode tornar determinadas aplicações economicamente inviáveis, exigindo técnicas de otimização como compressão de prompt proposto no artigo de Jiang et al. [17]
- **Latência:** Devido à complexidade $O(n^2)$ da atenção [5], contextos extensos aumentam drasticamente o tempo de *prefill* e geração, elevando a latência em aplicações interativas [18].

1.2 Problema de Pesquisa

Os avanços nas arquiteturas e nas técnicas de extensão de contexto ainda não conseguem superar a limitação da implementação de LLMs em cenários do dia a dia. Os principais modelos, que podem ser acessados por interface ou por API (como ChatGPT e Claude), impõem limites práticos ao comprimento de entrada e continuam sujeitos a alucinações, especialmente em contextos extensos. Para contornar essa limitação, torna-se comum a adoção de mecanismos de memória externa como (*Retrieval-Augmented Generation* RAG). Porém, na prática, mecanismos de busca vetorial aproximam semântica por distância em um espaço de embeddings, o

que não garante, em todos os casos, que os trechos recuperados sejam os semanticamente mais relevantes. Em Yu et al. [19] há a discussão mais ampla no limite das aplicações de RAGs.

Este trabalho parte da seguinte pergunta central:

Como projetar um assistente baseado em LLM capaz de (i) armazenar e reutilizar diálogos anteriores do usuário e (ii) recuperar informações factuais em fontes externas, de modo a construir contra-argumentações fundamentadas e inspiradas no método socrático?

Diferentemente de abordagens que modificam diretamente o mecanismo de atenção, os *position embeddings* ou que se limitam ao uso de um chat genérico, este estudo propõe um assistente que combina **memória externa persistente** e **recuperação de informações externas**. A memória registra e organiza o histórico de diálogos e o contexto específico do usuário, enquanto o módulo de recuperação externa consulta repositórios de documentos, notícias e outras fontes atualizadas para trazer evidências relevantes. Inspirado em trabalhos que exploram o método socrático em LLMs, seja para avaliação de raciocínios como no SocREval [20], seja para tutoria orientada por perguntas, como em modelos de ensino socrático [11, 21], o sistema aqui proposto utiliza essas informações recuperadas para formular perguntas e contra-argumentações baseadas em fatos, desempenhando um papel análogo ao *Elenque* na construção do diálogo socrático.

1.3 Objetivos

1.3.1 Objetivo Global

Desenvolver um *framework* não invasivo para LLMs que estenda a capacidade de processamento contextual por meio de memória externa persistente, recuperação de informações factuais e geração de contra-argumentações fundamentadas, inspirado no método socrático de refutação dialética.

1.3.2 Objetivos Específicos

1. Revisar sistematicamente as estratégias contemporâneas de extensão de contexto que operam **sem**:
 - (a) Retreinamento do modelo;
 - (b) Modificação dos mecanismos de atenção;

- (c) Alteração dos sistemas de *Position Embeddings*.
2. Projetar e implementar uma arquitetura de memória externa persistente para registro, indexação e reuso contextualizado do histórico de diálogos do usuário.
 3. Integrar mecanismo de recuperação de informações externas baseadas em RAG para fundamentar contra-argumentos em evidências documentais verificáveis.
 4. Elaborar, via raciocínio socrático, perguntas e refutações que testem premissas inconsistentes ou incompletas expressas pelo usuário, utilizando o histórico e as fontes externas recuperadas.
 5. Avaliar empiricamente a eficácia do *framework* em termos de:
 - Coerência e factualidade das respostas geradas;
 - Precisão na recuperação e uso do contexto histórico;
 - Qualidade e relevância das refutações socráticas produzidas.

1.4 Justificativa e Contribuições

A pesquisa sobre extensão de contexto em LLMs tem se concentrado predominantemente em intervenções arquiteturais diretas. Trabalhos como [5, 9] estabeleceram o mecanismo de auto-atenção como elemento central para captura de dependências de longo alcance, inspirando uma linha de pesquisa dedicada a modificações estruturais do Transformer.

Diversos surveys consolidam essa ênfase arquitetural, categorizando técnicas que alongam o contexto por meio de ajustes no Transformer, otimizações do mecanismo de atenção e esquemas especializados de codificação posicional [12, 22–27]. Embora essas abordagens demonstrem ganhos teóricos significativos, sua aplicação prática frequentemente esbarra em limitações de acesso: a maioria dos usuários de LLMs interage com modelos via API ou interfaces de chat, sem controle sobre a arquitetura interna ou capacidade de retreinamento.

Em contraste, este trabalho adota estratégias **não invasivas** — isto é, técnicas aplicáveis sobre modelos de API sem necessidade de retreinamento, fine-tuning ou modificação de componentes arquiteturais. Essas estratégias incluem compressão de prompts [28, 29], segmentação e processamento sequencial de janelas de contexto [30], arquiteturas de memória externa persistente [31, 32], e recuperação aumentada de informações externas (*Retrieval-Augmented Generation*, RAG) [19, 33, 34]. Trabalhos recentes têm refinado essas técnicas com abordagens como ganho de informação relevante para seleção de trechos [35], integração de grafos de conhecimento

esparços [36] e otimização de memória em tempo de execução [37]. Choi et al. [38] demonstram, por exemplo, que a combinação de RAG com adaptação de baixo-rank (LoRA) pode melhorar significativamente a precisão de recuperação enquanto reduz custos computacionais, evidenciando o potencial de técnicas não invasivas em aplicações especializadas.

Além disso, este trabalho propõe um diferencial metodológico: em vez de avaliar o sistema apenas por métricas de desempenho tradicionais (como perplexidade, BLEU ou taxa de acerto), introduz-se um enquadramento inspirado no **método socrático** [39–41]. Trabalhos recentes têm explorado essa abordagem em diferentes contextos: o SocREval [20] utiliza questionamento socrático para avaliação de raciocínio em LLMs sem referências externas, enquanto o SocraticLM [21] aplica ensino personalizado por meio de diálogos socráticos orientados. De forma complementar, este estudo propõe que o sistema não apenas armazene o histórico de interações do usuário, mas também integre fontes factuais externas via RAG para construir **contra-argumentações baseadas em evidências**, emulando o papel do *elenchos* (refutação) na tradição socrática.

Essa estratégia permite investigar, por meio de contraexemplos factuais e questionamento direcionado, os limites práticos de compreensão contextual em LLMs acessados via API — um cenário representativo do uso cotidiano desses modelos — e oferece um caminho para sistemas mais transparentes, verificáveis e pedagogicamente orientados.

1.4.1 Principais Contribuições

Este trabalho oferece as seguintes contribuições:

1. Uma revisão sistemática de técnicas **não invasivas** de extensão de contexto, categorizando abordagens aplicáveis sem acesso aos parâmetros ou arquitetura interna do modelo;
2. Uma arquitetura de **memória externa persistente** que organiza e reutiliza o histórico conversacional do usuário, combinada com **recuperação aumentada** de fontes documentais externas;
3. Um *framework* de **raciocínio socrático** que utiliza evidências recuperadas para gerar contra-argumentações fundamentadas, testando premissas e crenças inconsistentes expressas pelo usuário;
4. Uma avaliação empírica multidimensional que combina métricas tradicionais de qualidade textual com análise qualitativa da **relevância factual** e **eficácia pedagógica** das refutações produzidas.

1.5 Estrutura da Dissertação

Esta dissertação está organizada em cinco capítulos, estruturados de forma a conduzir o leitor desde os fundamentos teóricos até a avaliação prática do *framework* proposto:

Capítulo 1 – Introdução: Contextualiza o problema da janela de contexto em LLMs, apresenta os desafios práticos de custo, latência e degradação de desempenho em contextos extensos, formula a pergunta de pesquisa central, define os objetivos e justifica as contribuições esperadas. Estabelece o enquadramento metodológico inspirado no método socrático e situa o trabalho em relação às abordagens não invasivas de extensão de contexto.

Capítulo 2 – Fundamentos Técnicos e Teóricos: Apresenta os fundamentos filosóficos do método socrático (dialética, maiêutica e *elenchos*), com ênfase no uso de contraexemplos como ferramenta principal. Em seguida, revisa os princípios técnicos dos LLMs: arquitetura Transformer, mecanismo de atenção, complexidade computacional e funcionamento da janela de contexto. Discute sistematicamente as limitações práticas dos modelos em contextos longos, incluindo fenômenos como *context rot*, *lost in the middle* e colapso da atenção. Estabelece a base conceitual e técnica para as escolhas arquiteturais do *framework* proposto.

Capítulo 3 – Metodologia e *framework* Proposto: Descreve detalhadamente a arquitetura do sistema desenvolvido, incluindo: (i) o módulo de memória externa persistente para armazenamento e indexação de diálogos históricos; (ii) o mecanismo de recuperação aumentada (RAG) para acesso a fontes factuais externas; (iii) o módulo de raciocínio socrático que integra memória e recuperação para geração de contra-argumentações fundamentadas. Detalha os fluxos de execução, estratégias de implementação, algoritmos de indexação e recuperação, e o protocolo de interação socrática. Apresenta as especificações técnicas, decisões de design e justificativas para cada componente arquitetural.

Capítulo 4 – Avaliação Empírica e Discussão de Resultados: Apresenta os resultados obtidos a partir da avaliação do *framework* em cenários controlados e em uso real. Analisa a eficácia da arquitetura na coerência das respostas, na precisão da recuperação contextual e na qualidade das refutações socráticas produzidas. Discute os contraexemplos identificados, as limitações práticas observadas e os insights qualitativos sobre a interação entre memória, recuperação e raciocínio dialético. Integra análises quantitativas (métricas de desempenho, latência, custo) e qualitativas (relevância factual, eficácia pedagógica, transparência das refutações).

Capítulo 5 – Conclusão e Trabalhos Futuros: Síntese dos principais achados da pesquisa, reafirmação das contribuições e discussão das implicações práticas e teóricas do *framework* proposto. Identifica limitações metodológicas, desafios de escalabilidade e direções para investigações futuras, incluindo a integração com modelos multimodais, otimizações de latência e aplicações em domínios específicos como educação, saúde e análise jurídica.

Capítulo 2

Fundamentos Técnicos e Teóricos

2.1 Fundamentação Teórica: Método Socrático

2.1.1 O Método Socrático

O que hoje se conhece como *método socrático* não foi formalizado por Sócrates em um tratado sistemático, mas reconstruído a partir dos diálogos de Platão, nos quais Sócrates aparece como personagem central [3, 42, 43]. Nas obras de Platão podemos analisar o método não como uma *framework* rígida e definida, mas sim como um esilo de investigação baseado em perguntas bem elaboradas, diálogo rigorosamente crítico e exame das respostas para detecção de diversos elementos, como crenças limitantes.

Nas obras *Teeteto* e *Eutífron*, Platão apresenta Sócrates conduzindo interlocutores não como um mestre detentor da verdade absoluta, mas como um guia que conduzirá seus interlocutores em uma jornada de perguntas e respostas, em busca de definições que alcancem a *essência* do conhecimento [1] e [2]. Ao serem confrontados com exigências de coerência e com a exposição de contradições internas, os interlocutores trazem à tona o limite dos seus conhecimentos e crenças internas.

Em *Teeteto*, essa prática é exemplificada pela metáfora da **maiêutica** na qual Sócrates se apresenta como um “parteiro” de ideias. Incapaz de gerar doutrinas próprias, mas capaz de ajudar os outros a “dar à luz” a pensamentos que sejam frutos legítimos e não meros “falsos nascimentos”. Consolidando com isso a metodologia socrática como uma prática de liberdade filosófica, contrapondo a busca desinteressada pela verdade, aceitação a convenções sociais e crenças do comum e das opiniões coerentes.

A partir dessas leituras, é possível sintetizar o percurso central do diálogo socrático no fluxograma da Figura 2.1, que organiza, de forma esquemática, o movimento que vai da crença inicial do interlocutor à revisão de suas definições.

Dentro desse esquema, *Elenchos* corresponde ao componente de refutação do

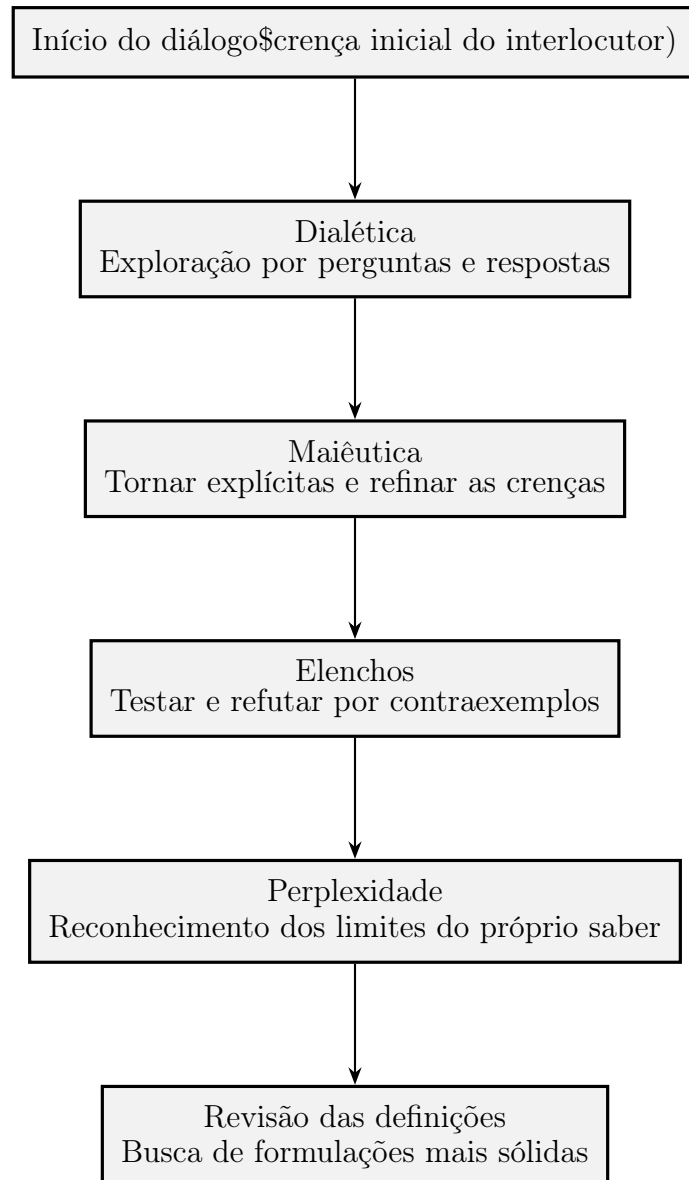


Figura 2.1: Fluxo das funções dialéticas, maiêuticas e elênticas no diálogo socrático [1–3].

método socrático. Trata-se de um procedimento de teste de hipóteses por meio de **contraexemplos**. Partindo de uma tese inicial do interlocutor, cujas implicações são examinadas até que se identifique (ou se construa) um caso concreto que a contradiga. Com isso, conduzindo o diálogo a um estado de perplexidade a partir do qual se torna necessário reformular a definição ou restringir sua validade.

É essa dinâmica que fundamenta o objetivo desta dissertação: projetar um assistente baseado em LLM capaz de orquestrar a recuperação de fatos em memória externa para gerar contraexemplos precisos. O sistema visa testar crenças inconsistentes do usuário, auxiliando-o na revisão de suas hipóteses em direção a respostas mais fundamentadas. Foi contruída uma tabela 2.1 para deixar mais claro as propostas

Tabela 2.1: Etapas do método socrático no contexto da dissertação

Pilar	Função no diálogo socrático	Papel na dissertação (assistente baseado em LLM)
Dialética	Conduzir o diálogo por meio de perguntas e respostas, explorando definições e implicações dos conceitos discutidos [1–3].	Estruturar a interação entre usuário e sistema como uma conversa guiada, em que o LLM explora e clarifica as crenças do interlocutor por meio de questionamentos sucessivos.
Maiêutica	“Partejar” ideias: ajudar o interlocutor a tornar explícitas suas crenças, refinando-as até que se tornem formuladas de forma clara e examinável [1].	Transformar opiniões vagas do usuário em hipóteses explícitas (enunciados claros) que possam ser testadas com fatos recuperados em memória externa e bases de dados.
Elenchos	Testar e refutar crenças por meio de contradições e contraexemplos, conduzindo o interlocutor a um estado de perplexidade e revisão de suas posições [1–3].	Utilizar informações factuais recuperadas externamente para gerar contraexemplos que desafiem crenças inconsistentes do usuário, estimulando a revisão das hipóteses e a aproximação a respostas mais bem fundamentadas.

2.1.2 Elenchos como refutação

Na interpretação clássica de Vlastos, o *elenchos* representa o núcleo lógico do método socrático. Ele traduz como sendo um procedimento de refutação que começa com uma afirmação do interlocutor e (através de perguntas adicionais) extrai consequências que acabam entrando em conflito com outras crenças que ele mesmo aceita [3]. O objetivo não é humilhar o interlocutor, mas demonstrar que o conjunto de crenças sustentado é internamente contraditório e convidando o interlocutor a suspender seu julgamento inicial e reconstruir suas ideias sobre bases mais sólidas. Em termos argumentativos, trata-se de uma refutação indireta. Onde assume-se provisoriamente a tese do interlocutor deriva-se um resultado inaceitável e a partir daí, conclui-se que pelo menos uma das premissas precisa ser revisada.

Esse procedimento de refutação apresenta duas características especialmente relevantes para esta dissertação. Em primeiro lugar, ele é **dialógico**, ou seja, a refutação não surge pronta mas sim é construída passo a passo a partir das próprias concessões do interlocutor. Em segundo lugar, ele é **interno** ao sistema de crenças examinado. Sócrates não recorre a autoridades externas e sim a princípios que o próprio participante aceita. O que torna a revisão conceitual epistemicamente mais robusta.

2.1.3 Contraexemplo como ferramenta epistemológica

Elenchos pode ser visto como um uso sistemático de **contraexemplos**. Diante de uma definição geral (por exemplo, “piedoso é aquilo que agrada aos deuses” em *Eutífron*) Sócrates propõe casos concretos que se encaixam na formulação, mas que, segundo outras crenças do interlocutor, não deveriam ser considerados piedosos [2]. Quando o interlocutor aceita o caso como legítimo, fica claro que a definição original era ampla demais ou restrita demais. Isso o obriga a ajustar ou substituir a formulação inicial [1, 2].

Esse modo de testar conceitos por meio de exemplos reaparece em contextos atuais. Ele está presente tanto na filosofia analítica quanto em aplicações clínicas do diálogo socrático, em que o terapeuta usa exemplos e contraexemplos para questionar crenças rígidas do paciente de forma conjunta, e não impositiva [41]. Nesses cenários, o contraexemplo tem dois papéis ao mesmo tempo. Do lado lógico, mostra que uma regra geral falha. Do lado pedagógico, torna essa falha visível para a própria pessoa que defendia a regra.

2.1.4 Adaptação do elenchos para avaliação de LLMs

Nesta dissertação, o *elenchos* não é usado para o modelo “refutar a si mesmo”, mas para questionar o usuário de forma estruturada. A ideia central é tratar o LLM como um facilitador de diálogo que identifica crenças ou afirmações potencialmente problemáticas e, em vez de apenas respondê-las, constrói perguntas e contraexemplos apoiados em fatos externos.

O fluxo é o seguinte. Primeiro, o usuário apresenta uma tese, opinião ou hipótese inicial em linguagem natural. O sistema então procura tornar explícito essa tese de uma forma que possa ser testada. Em seguida, módulos de memória (externa, ou técnicas de compressão) recupera informações de diferentes fontes de dados e também do próprio chat do usuário. Com base nesses dados, o LLM irá gerar perguntas, exemplos e contra exemplos que coloquem essa tese à prova. O objetivo não é ganhar o argumento, mas sim ajudar o usuário a ver onde suas crenças entram em choque com evidências apresentadas ou contra o próprio posicionamento prévio do usuário.

Do ponto de vista arquitetural, essas ferramentas de aumento da janela de contexto entram como uma ferramenta de suporte capaz de manter, em uma mesma interação, trechos relevantes de múltiplos documentos, resumos de partes passadas de conversa e anotações intermediárias produzidas pelo próprio sistema. O LLM acessa todo esse material para formular perguntas mais fundamentadas e contra exemplos mais específicos. Assim, o *elenchos* é adaptado como um padrão de interação onde o sistema usa recuperação em contexto longo para fundamentar questionamentos ao

usuário e guiá-lo na revisão de suas hipóteses, aproximando o diálogo computacional da prática socrática de refutação por contraexemplos.

2.2 Fundamentos Técnicos de LLMs

2.2.1 Modelos de Linguagem de Grande Escala

LLMs podem ser classificados como redes neurais profundas, baseadas na arquitetura Transformer [5] treinados para prever o próximo token em um grande *corpora* de texto. Matematicamente esses modelos aproximam-se a uma distribuição condicional $p(x_t | x_{<t})$, onde x_t representa o token atual e $x_{<t}$ denota todos os tokens anteriores a sequência [5, 9].

Diferente de modelos de rede neural especializados, que necessitam de treinamento e ajuste fino para conseguir realizar uma tarefa específica, LLMs funcionam como plataformas de propósito geral onde o próprio usuário, em linguagem natural, especifica o comportamento e ação desejada (como resumir documentos, responder perguntas, traduzir textos, analisar dados ou gerar código [9–11, 23]). Essa capacidade de generalização sem nenhum contexto é denominada como *zero-shot* ou *few-shot*. Isso só é possível por conta das alto volume e variação das fontes de dados que foram utilizados para treinamento dos modelos. Esses dados vieram de diversas origens (internet, livros, códigos, artigos), permitindo ao modelo capturar padrões linguísticos, estruturas semânticas e conhecimento contidos nesses documentos. A qualidade e diversidade dos dados de treinamento determina fundamentalmente a capacidade desses modelos generalistas em realizar tarefas específicas sem supervisão para os mais variados domínios.

2.2.2 Mecanismo de Atenção e Janela de Contexto

O mecanismo central dos LLMs é a **Atenção Multi-cabeças**, proposta por Vaswani et al. [5]. Seu funcionamento consiste no seguinte mecanismo: Para cada token de entrada, o mecanismo calcula pontos de atenção que quantificam o quanto esse token é importante para os demais tokens, permitindo capturar dependências textuais de forma otimizada e paralela.

Do ponto de vista matemático, cada token da sequência é projetado em 3 partes: *Consulta Q*; *Chave K*; *Valor V*. A auto atenção pode ser abstraída como um mecanismo que mede a similaridade entre a consulta e chaves e que em seguida usa essas pontuações como pesos para combinar os valores correspondentes. Formalmente, o mecanismo de atenção pode ser representado pela seguinte equação:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.1)$$

O termo QK^T produz uma matriz em que cada entrada representa a similaridade (via produto escalar) entre a consulta de um token e a chave de outro token. Com isso, cada linha dessa matriz indica o quanto um token considera relevantes todos os demais. O fator $\sqrt{d_k}$, onde d_k é a dimensão dos vetores de chave, é usado para normalizar esses produtos escalares e evitar valores muito grandes, que poderiam saturar a função de ativação e dificultar o treinamento [5]. Em seguida, a função $\text{softmax}()$ é aplicada para cada uma das linhas dessa matriz de similaridade para transformar cada uma delas em uma distribuição de probabilidades (ou seja, os pesos de atenção de um token sobre todos os outros somará 1). Por fim, essa matriz de probabilidades é multiplicado por V , de modo que cada token receba uma combinação linear dos vetores de valor de todos os tokens, ponderando então com a importância atribuída para cada um. O resultado é uma nova representação em que um token assume, de forma explícita, informações dos demais tokens considerados relevantes.

2.2.3 Complexidade Computacional Quadrática

Conforme já mencionado no Capítulo 1.1, o mecanismo de autoatenção apresenta complexidade $O(n^2 \cdot d)$. Essa complexidade decorre diretamente do cálculo da matriz de similaridade destrinchada na Secção 2.2.2. Do ponto de vista de implementação, essa matriz de atenção $n \times n$ precisa ser armazenada em memória durante o cálculo e cada camada do modelo (tipicamente entre 24 e 96 camadas em modelos de LLM modernos) mantém seu próprio *cache* de chaves e valores (*KV cache*) para evitar reprocessamento durante a autoregressão. O consumo total da memória cresce não apenas em n^2 , mas também com o número de camadas e o tamanho dos *chunks* ou *batches* de sequência processadas simultaneamente [37, 44].

As implicações práticas dessa complexidade incluem:

- **Limitação de memória em GPU:** em modelos com contexto muito longo, o armazenamento simultâneo das matrizes de atenção e ao longo de todas as camadas por rapidamente consumir a maior parte da memória disponível, exigindo cada vez mais VRAM disponível. Isso força o uso de estratégias como particionamento de cache, paginação de memória (*PagedAttention*) e gerenciamento fino de alocação para viabilizar o funcionamento de LLMs em servidores para produção [37].
- **Latência em inferência:** o tempo de processamento do *prefill* cresce de forma quadrática com n . Em cenários de contexto muito longo, essa etapa

pode prejudicar bastante a latencia do modelo, fazendo com que requisições que deveria demorar segundos, aumentarem bastante. Prejudicando com isso a experiência e usabilidade do usuário.

- **Custo operacional:** em ambientes de produção com precificação por interação, processar repetidamente grandes quantidades de contexto pode fazer com que o modelo se perca, priorizando partes que não são as principais para a interação, forçando com isso o usuário a repetir instruções ou enviar repetidas vezes a mesma informação, aumentando com isso os custos financeiros da aplicação. Isso motiva o uso de estratégias de compressão de prompt, seleção de trechos importantes e recuperação seletiva da informação, que reduzem com isso a necessidade de repetir interações [28].

Esses fatores reforçam a necessidade de buscar novas abordagens para processar todo o conteúdo da janela de contexto de uma forma mais eficiente. Como o objetivo deste estudo é desenvolver um *framework* agnóstico a modelos de LLM produtivos de mercado, sem necessidade de retreinamento ou implementação de arquiteturas customizadas, a seção seguinte revisa sistematicamente essas estratégias não invasivas, que operam sobre modelos já treinados sem modificações arquiteturais.

2.3 Estratégias Não-Invasivas para Extensão de Contexto

Por conta do problema descrito neste artigo de aumento de tokens na janela de contexto e o fato de que novos modelos não estão aumentando focando em evoluir no aumento da quantidade de tokens de input, uma variedade de técnicas não-invasivas vem surgindo para contornar essas restrições. Define-se como **técnicas não-invasivas** aquelas que operam **sem modificação do modelo base**, ou seja, sem retreinamento, sem acesso aos parâmetros internos, sem modificação das matrizes internas de peso e posição. A ideia dessa limitação neste artigo visa permitir a aplicação em modelos acessados via API e comercialmente mais utilizados. (como ChatGPT, Claude, Gemini ou Grok). Focando com então em desenvolver uma técnica agnóstica de gerenciamento de contexto e chat com o usuário e que possa ser facilmente replicada. Na literatura, poucos artigos tratam especificamente de aumento de janela de contexto não invasivos [12, 22].

Esta seção revê as principais técnicas de aumento da janela de contexto e também elabora uma proposta de taxonomia das principais estratégias identificadas na literatura. Essa classificação é organizada em quatro categorias principais conforme sua lógica operacional: (i) compressão de prompt, (ii) segmentação de contexto e

janela deslizante, (iii) métodos baseados em MapReduce, e (iv) alocação e gerenciamento de memória externa.

2.3.1 Compressão de Prompt

A técnica de **compressão de prompt** busca reduzir o número de tokens processados mantendo a semântica essencial do texto original e, consequentemente, minimizando custos computacionais sem sacrificar a qualidade das respostas [28, 29].

Jiang et al. [17, 28] propõem LLMingua e sua extensão LongLLMingua, que utilizam uma função de distorção combinada com penalização do comprimento do prompt (norma ℓ_0) para identificar e remover tokens redundantes mantendo relevância semântica. A abordagem baseia-se na minimização de:

$$\min_{\text{prompt comprimido}} \{ \text{Distorção}(\text{original}, \text{comprimido}) + \lambda \cdot \|\text{comprimido}\|_0 \},$$

onde o primeiro termo penaliza perda semântica e o segundo penaliza tamanho. Os experimentos demonstram **reduções de até 94%** no número de tokens, com preservação de contexto em torno de 92% [28].

Alternativamente, Gilbert et al. [29] compararam três abordagens de compressão: (i) básica (remoção de stopwords), (ii) semântica (seleção orientada por embeddings), e (iii) lossless (compressão sem perda informacional). Demonstraram que a compressão semântica, orientada apenas por instruções de prompt — sem modelo de compressão adicional —, consegue reduções similares com ganhos significativos em eficiência computacional.

Os benefícios práticos da compressão de prompt incluem:

- **Redução de custos:** diminui custos operacionais proporcionalmente ao número de tokens removidos, particularmente relevante em APIs com precificação por token;
- **Manutenção de qualidade:** preserva a semântica em torno de 92% através de seleção inteligente de tokens informativos [28];
- **Ideal para respostas rápidas:** reduz a carga computacional no prefill (pré-processamento), sendo apropriado para aplicações que demandam latência baixa.

2.3.2 Segmentação de Contexto e Janela Deslizante

A **segmentação de contexto** baseia-se na divisão de textos longos em segmentos menores que cabem individualmente na janela de contexto do modelo, processa-

dos em paralelo ou sequencialmente com estratégias de janelas deslizantes (*sliding windows*) [45].

Essa abordagem envolve quatro etapas principais:

1. **Particionamento:** divisão do documento longo em trechos com tamanho adequado à janela de contexto disponível;
2. **Processamento paralelo ou sequencial:** cada segmento é processado independentemente ou em sequência, dependendo dos recursos disponíveis;
3. **Preservação de continuidade:** a utilização de janelas sobrepostas (*overlapping windows*) ajuda a preservar a continuidade semântica entre segmentos sucessivos, evitando perda de informação nas fronteiras;
4. **Agregação:** resultados de cada segmento são combinados em uma resposta unificada, preservando coerência global.

A eficácia dessa abordagem depende criticamente de dois fatores: (i) correta definição dos limites dos segmentos, de modo a não quebrar dependências semânticas ou sintáticas importantes, e (ii) estratégia adequada de agregação dos resultados, que deve garantir a integridade e coerência do contexto global [45].

Ratner et al. [45] propõem o uso de **janelas paralelas de contexto**, em que múltiplos segmentos são processados simultaneamente e seus resultados combinados por mecanismos de votação ponderada ou fusão de representações. Os benefícios dessa abordagem incluem:

- **Inclusão de maior volume de dados:** permite processamento de textos que excedem significativamente a janela nominal do modelo;
- **Redução de custos:** em alguns cenários, pode reduzir custos ao permitir que apenas segmentos relevantes sejam processados em detalhe;
- **Adequação para textos multi-documento:** beneficia tarefas em que informação relevante está distribuída entre múltiplos documentos com conteúdo potencialmente redundante.

2.3.3 Métodos Baseados em MapReduce

Os **métodos MapReduce** fragmentam o texto em segmentos (fase Map), processam cada segmento individualmente, e em seguida agregam os resultados intermediários (fase Reduce), seguindo um paradigma de computação distribuída aplicado ao processamento de linguagem natural [30].

Kamradt et al. [30] descrevem e avaliam três variações principais dessa abordagem para contornar limitações de token em LLMs:

1. **Reduce Simple**s: agregação contínua de resumos e segmentos processados, concatenando resultados de forma sequencial. Cada segmento é sumarizado, e os resumos são progressivamente combinados até que caibam na janela de contexto final;
2. **QA Rank**: utiliza perguntas e respostas (*question answering*) para identificar os pontos principais de cada segmento que serão passados adiante na agregação, priorizando informação relevante à tarefa específica. Um conjunto de perguntas guia é aplicado a cada segmento, e apenas as respostas mais informativas são retidas para agregação;
3. **Refine**: agregação de resultados de forma hierárquica, em que outputs intermediários são refinados iterativamente até convergência. Cada etapa de agregação não apenas combina informações, mas as refina e melhora com base em critérios de qualidade definidos.

Esses métodos oferecem **estrutura modular para processamento e agregação de informações**, aumentando a precisão em tarefas de síntese e análise de textos extensos [30]. Os benefícios práticos incluem:

- **Modularidade**: permite composição flexível de etapas de processamento, facilitando customização para diferentes domínios e tarefas;
- **Escalabilidade**: pode ser aplicado recursivamente para textos arbitrariamente longos, sem limite teórico de comprimento;
- **Precisão em síntese**: particularmente eficaz para tarefas de sumarização multi-documento e análise comparativa em que informações complementares devem ser integradas.

2.3.4 Alocação e Gerenciamento de Memória Externa

A **memória externa** oferece uma abordagem fundamentalmente distinta das anteriores. Ao invés de tentar processar todo o conteúdo dentro da janela de contexto (comprimindo, segmentando ou agregando), o sistema mantém um repositório persistente de informações (histórico de diálogos, documentos indexados, grafos de conhecimento estruturados) e recupera dinamicamente apenas a informação mais relevante para cada consulta específica.

Esta categoria engloba diversas técnicas complementares que compartilham o princípio de externalizar o armazenamento de contexto e recuperá-lo sob demanda, conforme detalhado nas subseções a seguir.

MemGPT: LLMs como Sistemas Operacionais

MemGPT [31] é um *framework* inovador que integra LLMs com capacidades de memória hierárquica e gerenciamento de tarefas, fazendo com que o modelo funcione como um sistema operacional autônomo capaz de coordenar operações computacionais complexas. Caracteriza-se por três componentes principais:

- **Memória hierárquica:** distinção explícita entre memória de curto prazo (janela de contexto ativa do modelo) e memória de longo prazo (armazenamento persistente externo, tipicamente em banco de dados vetorial);
- **Gerenciamento automático:** o sistema determina dinamicamente quais informações manter em contexto ativo e quais arquivar na memória de longo prazo, utilizando heurísticas de relevância e recência;
- **Recuperação sob demanda:** busca inteligente de informações históricas baseada em similaridade semântica (via embeddings) e critérios de relevância contextual.

A arquitetura do MemGPT inspira-se diretamente em sistemas operacionais tradicionais, tratando a janela de contexto como análoga à memória RAM (limitada, rápida) e o armazenamento externo como análogo ao disco (ilimitado, mais lento). O sistema decide autonomamente quando fazer “paginação” de informações entre os dois níveis [31].

InfLLM: Memória de Contexto Eficiente

InfLLM [32] propõe um método de extrapolação de contexto longo sem treinamento adicional, baseado na utilização de uma memória de contexto eficiente que organiza e armazena informações de maneira escalável. A abordagem distingue-se por três características:

- **Armazenamento estruturado:** informações do contexto são segmentadas e indexadas em estruturas de dados otimizadas para busca rápida;
- **Recuperação seletiva:** apenas os segmentos mais relevantes para a consulta atual são injetados na janela de contexto, reduzindo drasticamente a carga computacional;
- **Manutenção de coerência:** mecanismos de controle garantem que dependências de longo alcance sejam preservadas mesmo quando informações intermediárias não estão ativas no contexto.

O InfLLM possibilita que o modelo mantenha coerência e qualidade na geração de texto mesmo ao processar entradas extremamente longas (potencialmente ilimitadas), sem necessidade de retreinamento ou modificação arquitetural [32].

Relevant Information Gain (RIG)

Pickett et al. [35] propõem uma melhoria nos métodos de RAG (*Retrieval-Augmented Generation*) ao incorporar o conceito de **ganho de informação relevante** para otimizar a seleção e integração dos dados recuperados.

A abordagem tradicional de RAG recupera documentos baseando-se apenas em similaridade semântica (distância em espaço de embeddings), o que não garante que os trechos recuperados sejam os mais *informativos* para responder à consulta. O RIG introduz uma métrica que quantifica o quanto cada documento recuperado contribui para reduzir a incerteza sobre a resposta correta, priorizando trechos verdadeiramente informativos em detrimento de trechos semanticamente similares mas redundantes ou irrelevantes.

Formalmente, dado um conjunto de documentos candidatos $\{d_1, d_2, \dots, d_k\}$, o RIG calcula:

$$\text{RIG}(d_i) = H(Y) - H(Y \mid d_i),$$

onde $H(Y)$ é a entropia da distribuição de respostas antes de observar o documento, e $H(Y \mid d_i)$ é a entropia condicional após observar d_i . Documentos que maximizam o ganho de informação são prioritariamente incluídos no contexto [35].

PagedAttention: Otimização de Memória

PagedAttention [37] aborda especificamente o problema de gerenciamento de memória durante a inferência de LLMs, propondo uma técnica que segmenta o cache de atenção (KV cache) em “páginas”, analogamente a sistemas de paginação em memória virtual de sistemas operacionais.

Durante a geração autoregressiva, o modelo precisa armazenar os vetores *key* e *value* de todos os tokens previamente gerados para computar a atenção eficientemente. Em sequências longas, esse cache consome memória significativa. O PagedAttention:

- **Segmenta o cache:** divide o KV cache em blocos (páginas) de tamanho fixo;
- **Aloca dinamicamente:** páginas são alocadas sob demanda conforme a geração avança, reduzindo desperdício de memória;
- **Permite compartilhamento:** múltiplas sequências geradas em paralelo (*beam search*, *batch inference*) podem compartilhar páginas comuns, economizando memória.

Benefícios incluem [37]:

- **Melhor escalabilidade:** permite processar sequências maiores ou mais sequências em paralelo com o mesmo hardware;
- **Redução de sobrecarga:** diminui desperdício de memória causado por alocação inflexível;
- **Preservação de dependências:** mantém qualidade das representações apesar da fragmentação do cache.

Knowledge Graph Extraction

A abordagem de extração de grafos de conhecimento propõe a transformação de textos longos e não estruturados em representações estruturadas baseadas em grafos, nas quais nós representam entidades ou conceitos, e arestas representam relações entre eles [36]. O processo de criação de grafos de conhecimento divide-se em três etapas principais:

1. **Extração de informações relevantes:** identificação de entidades (pessoas, organizações, locais, conceitos) e relações (causais, temporais, hierárquicas) presentes no texto;
2. **Definição de conceitos e entidades:** estruturação semântica que organiza as entidades extraídas em uma ontologia ou taxonomia coerente;
3. **Canonização:** padronização dos dados extraídos para resolver ambiguidades (entidades diferentes com nomes similares, ou a mesma entidade referenciada de formas distintas).

Essa metodologia permite **compactação de informações em estruturas ordenadas**, reduzindo significativamente o espaço de armazenamento necessário enquanto mantém relações semânticas explícitas e navegáveis [36]. Ao invés de manter o texto completo em memória, o sistema armazena apenas o grafo de conhecimento e, quando necessário, recupera subgrafos relevantes para injetar no contexto.

Sparse GraphRAG e Mixture-of-PageRanks

Alonso e Millidge [36] apresentam uma abordagem inovadora denominada Mixture-of-PageRanks, que substitui a dependência de longos contextos em LLMs por um mecanismo de **grafos esparsos em tempo real**, inspirado no algoritmo PageRank utilizado originalmente para ranqueamento de páginas web.

Em vez de manter toda a informação na janela de contexto ou mesmo em memória densa, o sistema:

- **Constrói grafos de entidades e relações:** representação estruturada do conhecimento contido em documentos longos ou coleções de documentos, onde a estrutura do grafo captura dependências e relevância relativa;
- **Calcula relevância dinâmica:** usa variantes do algoritmo PageRank para identificar nós (entidades/conceitos) mais importantes no contexto da consulta atual, ponderando importância global (centralidade no grafo) e relevância local (proximidade semântica à consulta);
- **Recupera dinamicamente:** injeta na janela de contexto apenas as partes do grafo (subgrafos) relevantes à consulta, com suas conexões explícitas preservadas.

A abordagem denomina-se “Mixture-of-PageRanks” porque combina múltiplas funções de ranqueamento aplicadas simultaneamente ao mesmo grafo, capturando diferentes dimensões de relevância (temporal, causal, topológica) [36].

Benefícios desta abordagem incluem:

- **Escalabilidade:** pode processar bases de conhecimento de tamanho arbitrário, limitada apenas pela capacidade de armazenamento do grafo (não pela janela de contexto);
- **Eficiência:** reduz drasticamente o número de tokens necessários, mantendo apenas informações estruturalmente relevantes no contexto ativo;
- **Rastreabilidade:** grafos explícitos tornam as fontes de informação transparentes e auditáveis, facilitando verificação e mitigando alucinações.

2.3.5 Síntese: Taxonomia de Técnicas Não-Invasivas

A Tabela 2.2 oferece uma visão consolidada das técnicas revisadas, organizadas por categoria operacional, com indicação de seus mecanismos principais e aplicações ideais.

2.3.6 Avaliação de Eficácia: Métricas e Resultados

A eficácia das técnicas de extensão de contexto é avaliada segundo múltiplas dimensões complementares, conforme documentado na literatura revisada [38, 46, 47].

Redução de Tokens e Preservação Semântica

Técnicas de compressão demonstram capacidade significativa de redução de tokens mantendo qualidade:

Tabela 2.2: Taxonomia de técnicas não-invasivas para extensão de contexto em LLMs

Categoria	Técnicas	Mecanismo Principal	Aplicação Ideal
Compressão de Prompt	LLMLingua, LongLLMLingua, Semantic Compression	Remoção de tokens redundantes com preservação semântica	Respostas rápidas, redução de custo, APIs com limitação de tokens
Segmentação de Contexto	Janela Deslizante, Janelas Paralelas	Divisão em trechos + processamento paralelo/sequencial + agregação	Textos multi-documento, processamento paralelo em GPU
Métodos MapReduce	Reduce Simples, QA Rank, Refine	Fragmentação (Map) + processamento + agregação hierárquica (Reduce)	Síntese e análise de textos extensos, sumarização multi-documento
Memória Externa	MemGPT, InLLM, RIG, PagedAttention, Knowledge Graphs, Sparse GraphRAG	Armazenamento persistente + recuperação dinâmica baseada em relevância	Diálogos prolongados, bases de conhecimento extensas, análises detalhadas

- Compressão de prompt pode reduzir em até **94% o número de tokens** processados [28];
- Abordagens de reconstrução semântica preservam o contexto em torno de **92%**, medido por similaridade de embeddings e métricas de coerência [28];
- Mesmo com reduções agressivas, tarefas de question answering e sumarização mantêm qualidade comparável ao processamento sem compressão [29].

Latência e Custo Computacional

Ganhos em eficiência computacional são documentados em múltiplos trabalhos:

- Inferência até **2.6x mais rápida** com uso de compressão de prompt, devido à redução no tempo de prefill [48];
- Menor utilização de memória GPU e redução proporcional dos custos operacionais em ambientes de produção [48];
- Técnicas de LoRA (*Low-Rank Adaptation*) aplicadas a RAG reduzem parâmetros treináveis em **13.6%** e custos computacionais em **18.7%** [38].

Performance em Tarefas Específicas

Avaliações em benchmarks especializados demonstram melhorias consistentes:

- Melhor desempenho em tarefas de *question answering*, extração de informações e sumarização, particularmente quando informações relevantes estão distribuídas em múltiplos documentos [46];
- Métrica BLEU-4 aumenta em **4.36%**, CIDEr em **6.83%**, e SPICE em **5.46%** com abordagens otimizadas de RAG combinadas com LoRA [38];
- Precisão de recuperação (*retrieval accuracy*) melhora em **8.5%** comparado a sistemas RAG baseline, quando técnicas de ganho de informação relevante são aplicadas [38].

2.3.7 Benchmarks Especializados para Contexto Longo

A avaliação rigorosa de capacidades de longo contexto requer benchmarks que testem não apenas a capacidade técnica de processar sequências longas, mas o uso *efetivo* da informação distribuída ao longo do contexto [46, 47]:

- **LongBench** [46]: benchmark bilíngue e multitarefa para compreensão de contexto longo, incluindo tarefas de question answering, sumarização e análise de código com sequências de até 16k tokens. Avalia capacidade de recuperação, síntese e raciocínio sobre informações dispersas;
- **RULER** [47]: *framework* que mede o “tamanho real de contexto” de LLMs através de testes de recuperação de informação inserida em posições variadas ao longo da sequência, expondo quantitativamente fenômenos como “lost in the middle” e degradação posicional;
- **Needle-in-a-Haystack**: teste simples mas revelador no qual um fato específico (“agulha”) é inserido em posição aleatória em um contexto longo (“palheiro”), e o modelo deve recuperá-lo com precisão. Expõe vieses posicionais, degradação de atenção e limitações práticas de janelas nominalmente grandes.

Esses benchmarks revelam sistematicamente que janelas de contexto nominalmente grandes (100k, 1M+ tokens) frequentemente não são utilizadas de forma eficaz pelo modelo, com desempenho degradando significativamente quando informação crítica está além dos primeiros 32-64k tokens ou posicionada em regiões centrais da sequência [16, 46, 47].

2.3.8 Síntese e Lacunas Identificadas

Os trabalhos revisados demonstram ganhos significativos em eficiência e desempenho utilizando apenas técnicas não invasivas em modelos. Porém parte dos métodos de teste são inconsistentes ou muito básicos para avaliar a performance de modelos na vida real. Com isso, no capítulo seguinte iremos realizar testes e avaliações nos métodos mais promissores e detalhar também a metodologia aplicada no desenvolvimento da *framework* de contra exemplos do Elenque.

Capítulo 3

Metodologia

Este capítulo descreve a arquitetura do *framework* proposto, os procedimentos experimentais para sua avaliação e as métricas utilizadas para medir sua eficácia. A metodologia basea-se em três principais etapas:

- Validação empírica das técnicas não invasivas revisadas no Capítulo ??, com seleção da abordagem que apresentar melhor desempenho segundo *benchmarks* especializados;
- Implementação de um *framework* socrático com memória externa persistente, focado em recuperar informações externas ao treinamento do modelo e armazenar o histórico conversacional para busca de contraexemplos;
- Avaliação qualitativa e quantitativa da interação dialética produzida.

3.1 Validação Empírica de Técnicas Não-Invasivas

Para selecionar a melhor estratégia de extensão de **janela de contexto** para a *framework* proposta, conduzimos uma avaliação de desempenho das técnicas revisadas na Seção 2.3, utilizando três *benchmarks* especializados em contexto longo: Needle-in-a-Haystack, RULER e LongBench, conforme justificado também na Seção 3.1. Os experimentos foram conduzidos utilizando imagens **docker**, código em python e utilização de API GROQ com quatro modelos principais : GPT OSS 120B, GPT OSS 20B, Llama 3.1 8B e Llama 3.3 70B.

3.1.1 Estratégias de Avaliação

As seguintes estratégias foram adotadas para os testes:

- **Ras (Baseline)**: processamento direto do contexto completo (até o limite), sem técnica de otimização;

- **Parallel Window**: processo de segmentação em janelas paralelas processadas simultaneamente, seguida de agregação dos resultados como descrito em [45];
 - **Sliding Window**: janela deslizante com sobreposição para preservar continuidade semantica, como descrito em [30];
 - **RIG (Dartboard)**: recuperação de informação através de memória externa, priorizando ganho da informação relevante [35];
- Semantic Compression**: compressão de *prompt* orientada por preservação semântica [29].

3.1.2 Resultados: Acurácia e Latencia

As Tabelas 3.1 e 3.2 apresentam, respectivamente, as pontuações de acurácia (normalizados entre 0 e 1) e a latência média (em milissegundos ou segundos) obtidas por cada estratégia nos 3 *benchmarks* selecionados.

Tabela 3.1: Acurácia por estratégia nos *benchmarks* de contexto longo

Modelo	Estratégia	Needle	RULER	LongBench	Média
Llama 3.1 8B	Parallel Window	1.00	1.00	0.94	0.98
	Raw (Baseline)	1.00	0.94	0.94	0.96
	RIG (Dartboard)	1.00	1.00	0.94	0.98
	Semantic Compression	0.33	0.56	0.94	0.61
	Sliding Window	1.00	1.00	0.94	0.98
Llama 3.3 70B	Parallel Window	1.00	1.00	0.94	0.98
	Raw (Baseline)	1.00	1.00	1.00	1.00
	RIG (Dartboard)	1.00	1.00	0.94	0.98
	Semantic Compression	0.33	0.69	0.38	0.47
	Sliding Window	1.00	1.00	0.94	0.98
GPT-OSS 20B	Parallel Window	1.00	0.69	—	0.85
	Raw (Baseline)	1.00	0.78	0.50	0.76
	RIG (Dartboard)	1.00	0.50	—	0.75
	Semantic Compression	0.73	0.00	—	0.37
	Sliding Window	1.00	0.72	—	0.86
GPT-OSS 120B	Parallel Window	1.00	0.58	0.00	0.53
	Raw (Baseline)	1.00	0.56	0.17	0.58
	RIG (Dartboard)	0.97	0.44	—	0.71
	Semantic Compression	1.00	0.06	—	0.53
	Sliding Window	0.87	0.64	0.00	0.50

Tabela 3.2: Latência média por estratégia (ms)

Modelo	Estratégia	Needle	RULER	LongBench	Média
Llama 3.1 8B	Parallel Window	6200	3700	2400	4100
	Raw (Baseline)	318	1000	344	554
	RIG (Dartboard)	6300	3900	6300	5500
	Semantic Compression	11700	7300	11700	10200
	Sliding Window	2300	3800	286	2129
Llama 3.3 70B	Parallel Window	271	2300	346	972
	Raw (Baseline)	247	280	316	281
	RIG (Dartboard)	3200	2500	1500	2400
	Semantic Compression	3600	5100	965	3222
	Sliding Window	262	517	336	372
GPT-OSS 20B	Parallel Window	7134	3489	—	5312
	Raw (Baseline)	2775	298	2766	1946
	RIG (Dartboard)	7055	2766	—	4911
	Semantic Compression	14676	130	—	7403
	Sliding Window	7527	3399	—	5463
GPT-OSS 120B	Parallel Window	7165	3372	159	3565
	Raw (Baseline)	3507	2141	231	1960
	RIG (Dartboard)	7272	3085	—	5179
	Semantic Compression	16670	2071	—	9371
	Sliding Window	7681	3638	71	3797

3.1.3 Discussão dos Resultados e Seleção da Estratégia

Os resultados indicam um fator inesperado. Em modelos maiores, por se tratar de benchmark publicos, os modelos de aumento da janela de contexto tem uma acurácia melhor que simplesmente usar a janela de contexto por si só. Já nos modelos menores, com menos parametros de treinamento, as metodologias de **Parallel Window**, **RIG** e **Sliding Window** obtiveram acurácia equivalente (0.98), mas **Sliding Window** destacou-se pela menor latência (2.2s).

3.2 Arquitetura do Framework Prospoto

Framework de Diálogo com Extensão de Contexto

RIG (Dartboard Ranking) + Compressão Adaptativa

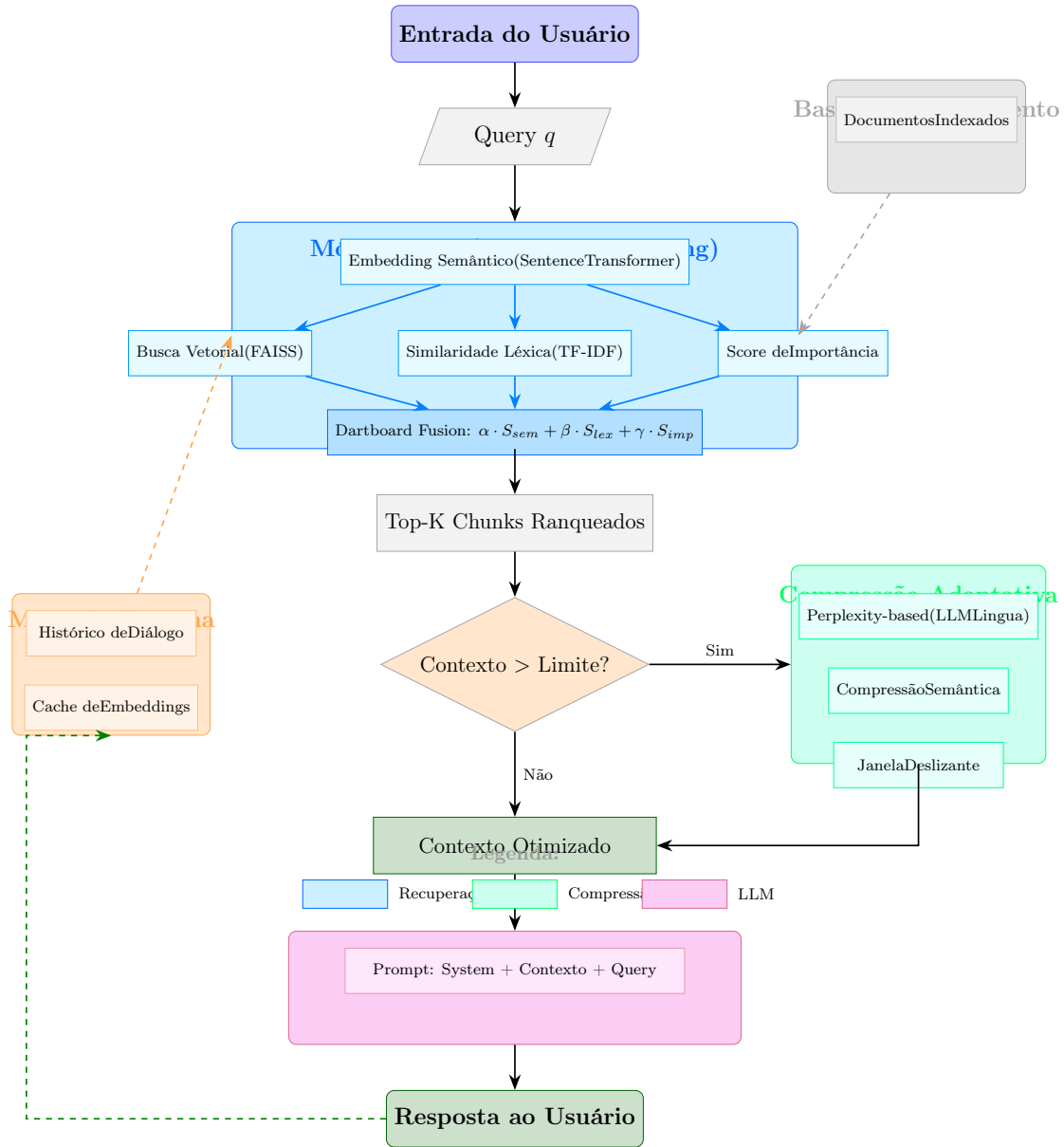


Figura 3.1: Diagrama de fluxo do framework de diálogo com extensão de contexto. O sistema integra recuperação via RIG (Dartboard Ranking), compressão adaptativa e geração via LLM.

Capítulo 4

Resultados e Discussões

Capítulo 5

Conclusão

Referências Bibliográficas

- [1] PLATO. *Theaetetus*. Indianapolis, Hackett Publishing, 1990.
- [2] PLATO. *Euthyphro*. Indianapolis, Hackett Publishing, 1997. In: Plato, Complete Works.
- [3] Vlastos, G. (Ed.). *Socratic Studies*. Cambridge University Press, 1994.
- [4] MIKOLOV, T., CHEN, K., CORRADO, G., et al. “Efficient Estimation of Word Representations in Vector Space”, *arXiv preprint arXiv:1301.3781*, 2013.
- [5] VASWANI, A., SHAZEER, N., PARMAR, N., et al. “Attention is all you need”. In: *Advances in neural information processing systems*, v. 30, 2017.
- [6] DEVLIN, J., CHANG, M., LEE, K., et al. “BERT: Pre-training of Deep Bi-directional Transformers for Language Understanding”, pp. 4171–4186, 2019. doi: 10.18653/V1/N19-1423. Disponível em: <<https://doi.org/10.18653/v1/n19-1423>>.
- [7] RADFORD, A., NARASIMHAN, K., SALIMANS, T., et al. “Improving Language Understanding by Generative Pre-Training”, 2018.
- [8] RUDER, S., PETERS, M. E., SWAYAMDIPTA, S., et al. “Transfer Learning in Natural Language Processing”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Tutorials*, pp. 15–18, 2019.
- [9] BROWN, T. B., MANN, B., RYDER, N., et al. “Language Models are Few-Shot Learners”, *Advances in Neural Information Processing Systems*, v. 33, pp. 1877–1901, 2020.
- [10] HUYNH, N., LIN, B. “Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications”, *arXiv preprint arXiv:2503.01245*, 2025. doi: 10.48550/arXiv.2503.01245. Disponível em: <<https://arxiv.org/abs/2503.01245>>.

- [11] SCARLATOS, A., LIU, N., LEE, J., et al. “Training LLM-Based Tutors to Improve Student Learning Outcomes in Dialogues”. In: *Artificial Intelligence in Education*, p. 251–266, Springer Nature Switzerland, 2025. ISBN: 9783031984143. doi: 10.1007/978-3-031-98414-3_18. Disponível em: <http://dx.doi.org/10.1007/978-3-031-98414-3_18>.
- [12] HUANG, Y., XU, J., LAI, J., et al. “Advancing Transformer Architecture in Long-Context Large Language Models: A Comprehensive Survey”. 2024. Disponível em: <<https://arxiv.org/abs/2311.12351>>.
- [13] REID, M., SAVINOV, N., TEPLYASHIN, D., et al. *Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context*. Technical report, Google DeepMind, 2024.
- [14] CHROMA. “The curse of the long context: Why your RAG system is failing”. <https://www.trychroma.com/blog>, 2024. Accessed: 2024-05-20.
- [15] CHEN, S., LIN, Z., POLYANSKIY, Y., et al. “Critical attention scaling in long-context transformers”. 2025. Disponível em: <<https://arxiv.org/abs/2510.05554>>.
- [16] LIU, N. F., LIN, K., HEWITT, J., et al. “Lost in the middle: How language models use long contexts”, *Transactions of the Association for Computational Linguistics*, v. 12, pp. 157–173, 2024.
- [17] JIANG, H., WU, Q., LIN, C.-Y., et al. “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13358–13376, 2023.
- [18] AMAN, S., RESEARCHERS, A. “Latency Analysis in Long-Context LLM Applications”, *AI Performance Review*, 2024.
- [19] YU, T., XU, A., AKKIRAJU, R. “In Defense of RAG in the Era of Long-Context Language Models”. 2024. Disponível em: <<https://arxiv.org/abs/2409.01666>>.
- [20] HE, H., ZHANG, H., ROTH, D. “Socreval: Large language models with the socratic method for reference-free reasoning evaluation”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [21] LIU, J., OTHERS. “SocraticLM: Exploring Socratic Personalized Teaching with Large Language Models”. In: *Advances in Neural Information*

Processing Systems, 2024. Disponível em: <<http://staff.ustc.edu.cn/~huangzhy/files/papers/JiayuLiu-NeurIPS2024.pdf>>. NeurIPS 2024, preprint available online.

- [22] WANG, X., SALMANI, M., OMIDI, P., et al. “Beyond the Limits: A Survey of Techniques to Extend the Context Length in Large Language Models”. 2024. Disponível em: <<https://arxiv.org/abs/2402.02244>>.
- [23] ZHAO, W. X., ZHOU, K., LI, J., et al. “A Survey of Large Language Models”, *arXiv preprint arXiv:2303.18223*, 2023.
- [24] NAVEED, H., KHAN, A., IQBAL, M., et al. “A Comprehensive Overview of Large Language Models”, *arXiv preprint arXiv:2307.06435*, 2023.
- [25] WAN, Z., WANG, X., ZHANG, J., et al. “Efficient Large Language Models: A Survey”, *arXiv preprint arXiv:2312.03863*, 2023.
- [26] HUANG, Y., XU, J., WANG, Y., et al. “Advancing Transformer Architecture in Long-Context Large Language Models: A Comprehensive Survey”, *arXiv preprint arXiv:2311.12351*, 2023.
- [27] ZHUANG, B., LIU, J., PAN, Z., et al. “A Survey on Efficient Training of Transformers”. 2023. Disponível em: <<https://arxiv.org/abs/2302.01107>>.
- [28] JIANG, H., WU, Q., LUO, X., et al. “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression”. 2024. Disponível em: <<https://arxiv.org/abs/2310.06839>>.
- [29] GILBERT, H., SANDBORN, M., SCHMIDT, D. C., et al. “Semantic Compression With Large Language Models”. 2023. Disponível em: <<https://arxiv.org/abs/2304.12512>>.
- [30] KAMRADT, G. “Workaround OpenAI’s Token Limit With Chain Types”. https://www.youtube.com/watch?v=f9_BWhCI4Zo&t=319s, 2023. Acessado em: 25-Jan-2025.
- [31] PACKER, C., WOODERS, S., LIN, K., et al. “MemGPT: Towards LLMs as Operating Systems”. 2024. Disponível em: <<https://arxiv.org/abs/2310.08560>>.
- [32] XIAO, C., ZHANG, P., HAN, X., et al. “InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory”. 2024. Disponível em: <<https://arxiv.org/abs/2402.04617>>.

- [33] LEWIS, P., PEREZ, E., PIKTUS, A., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*, v. 33, pp. 9459–9474, 2020.
- [34] SHARMA, C. “Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers”. 2025. Disponível em: <<https://arxiv.org/abs/2506.00054>>.
- [35] PICKETT, M., HARTMAN, J., BHOWMICK, A. K., et al. “Better RAG using Relevant Information Gain”, *arXiv preprint*, 2024.
- [36] ALONSO, N., MILLIDGE, B. “Mixture-of-PageRanks: Replacing Long-Context with Real-Time, Sparse GraphRAG”, *arXiv preprint*, 2024.
- [37] KWON, W., LI, Z., ZHUANG, S., et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention”, *arXiv preprint*, 2023.
- [38] CHOI, Y., KIM, S., BASSOLE, Y. C. F., et al. “Enhanced Retrieval-Augmented Generation Using Low-Rank Adaptation”, *Applied Sciences*, v. 15, n. 8, 2025. ISSN: 2076-3417. doi: 10.3390/app15084425. Disponível em: <<https://www.mdpi.com/2076-3417/15/8/4425>>.
- [39] VLASTOS, G. *Socratic studies*. Cambridge University Press, 1994.
- [40] PLATO. *Theaetetus*. Hackett Publishing, 2015. Translated by M. J. Levett, Revised by M. Burnyeat.
- [41] MONTAZERI, M., OTHERS. “The Socratic Method in Psychotherapy and Counseling: A Theoretical Analysis”, *Journal of Rational-Emotive & Cognitive-Behavior Therapy*, 2022.
- [42] Platão (Ed.). *teeteto*. Fundação Calouste Gulbenkian, 2015.
- [43] PETRELLI, H. Z. “PLATÃO Eutífron”, *Sofia*, v. 2, n. 1, 2013.
- [44] MIAO, X., OLIARO, G., ZHANG, Z., et al. “Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems”, *arXiv preprint arXiv:2312.15234*, 2024. Survey on efficient LLM serving including memory constraints and KV cache.
- [45] RATNER, N., LEVINE, Y., BELINKOV, Y., et al. “Parallel Context Windows for Large Language Models”. 2023. Disponível em: <<https://arxiv.org/abs/2212.10947>>.

- [46] BAI, Y., LV, X., ZHANG, J., et al. “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding”, *arXiv preprint arXiv:2308.14508*, 2023.
- [47] HSIEH, C.-P., LI, S., YEH, C.-K., et al. “Ruler: What is the real context size of your long-context language models?” *arXiv preprint arXiv:2404.06654*, 2024.
- [48] MIAO, X., OLIARO, G., ZHANG, Z., et al. “Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems”, *arXiv preprint arXiv:2312.15234*, 2024. Review on efficiency including context memory constraints.